Quantum-Assisted Knowledge Distillation for Lightweight Waste Classification: A Comparative Study

Author names
Affiliation

***Abstract*—Knowledge distillation (KD) suits camera-assisted waste sorting on low-power hardware, because only the compact student is deployed. A recent proposal adds a quantum kernel to the distillation loss, so that quantum computation is needed only during training. Testing whether such a kernel helps beyond kernel alignment itself requires matched classical controls. We run this test on TrashNet, with RPS-25 as a low-data confirmation, using a ResNet-50 teacher, three students and one shared pipeline. Hard-label training, logit KD, relational KD, a classical RBF-kernel twin and a projected quantum kernel (PQK) are compared with three seeds per cell. PQK is not distinguishable from logit KD. It is ahead on the mean in 3 of 6 student–dataset pairs and behind on both TrashNet MobileNets, and no paired difference is significant (smallest uncorrected $p = 0.22$). At its pre-specified kernel scale it beats its RBF twin in only 2 of 6 pairs. Its seed-to-seed variance tends to be lower than the twin's, though not significantly. All 66 exported MobileNet students are fully integer INT8 models of about 0.62 and 1.21 MB that run in 2.5 and 3.2 ms on a Raspberry Pi 4B, and the INT8 kernels used on the device change each method's measured effect. The quantum kernel leaves no trace in the deployed model, but our exploratory evidence does not show an accuracy benefit over classical alternatives.**

I. Introduction

Sorting waste at the point of disposal, for example in a camera-assisted bin that suggests a category for the item presented to it, needs a classifier that runs locally on inexpensive hardware with a small memory footprint and low latency. The available data are small and imbalanced. TrashNet [1] has six material classes, with 96 to 416 training images per class in our split. Compact networks such as MobileNets [2, 3] and 8-bit integer inference [4] reduce the footprint at some cost in accuracy.

Knowledge distillation (KD) [5] recovers part of that accuracy. The compact student is trained to match a larger teacher's outputs as well as the hard labels. Relational variants transfer the geometry of the teacher's representation instead [6, 7]. Because only the student is deployed, distillation combines naturally with quantization on edge devices [8].

Recent work asks whether quantum computation can assist distillation while the deployed model stays classical. Quantum relational knowledge distillation (QRKD) [9] computes quantum-kernel values between features mapped into a Hilbert space and uses them as relational targets. It reports gains over classical relational KD, with quantum computation needed only in training. Other work distills classical teachers into quantum students [10, 11] or uses a quantum model as a soft-label teacher [12]. We ask a narrower question: does the quantum component contribute anything that a classical kernel in the same place would not?

For kernels, the answer is not obvious. Quantum kernel values can concentrate until they no longer distinguish inputs [13]. Numerical studies find no advantage for fidelity kernels on classical data [14], and bandwidth-tuned quantum kernels can resemble classical ones [15]. The projected quantum kernel (PQK) of Huang et al. [16] is a clean candidate to test. At small scale it can be simulated classically, and a classical kernel acting on the same inputs can replace it one-for-one. Two further points shape the comparison. Effects on small datasets are comparable to seed-to-seed noise, so conclusions need several seeds and paired comparisons. The end point is the deployed INT8 model, and its results depend on the kernels that execute it.

We therefore compare quantum-assisted and classical distillation under one pipeline. TrashNet is the primary dataset. RPS-25, a 25% training subset of Rock–Paper–Scissors [17], serves as a low-data confirmation. A frozen ResNet-50 teacher [18] distills into three students of 0.03 M to 0.93 M parameters. We compare five methods: hard-label training (`scratch`), logit KD (`kd`), relational KD (`rkd`), classical RBF-kernel alignment (`rbf_control`, the matched twin) and PQK (`pqk`). Each cell has 3 seeds, and every exported model is a plain INT8 network with no quantum operation.

Our findings are as follows.

- `pqk` is not distinguishable from `kd`. It is ahead on the mean in 3 of 6 pairs and behind on both TrashNet MobileNets (by 0.025 and 0.014 macro-F1). The smallest uncorrected paired p-value is 0.22.
- At its pre-specified kernel scale, `pqk` beats its RBF twin in 2 of 6 pairs. With a median-heuristic scale adopted after these results were seen, it does so in 3 of 4, and the one nominally significant difference favours the classical twin.
- `pqk` tends to have lower seed-to-seed variance than `rbf_control` and, less consistently, than `scratch`. The difference is not significant at three seeds.
- All 66 exported MobileNet students are fully integer INT8 models. INT8 conversion changes macro-F1 by +0.003 on average, and the size of each method's INT8 effect depends on the runtime kernels.

All comparisons are exploratory. Cells have three seeds and the test splits informed earlier design choices, so we select on validation and treat test scores as descriptive.

We make three contributions. (i) A matched-control comparison of quantum-assisted and classical distillation for lightweight waste classification, with a classical twin for the quantum kernel and paired per-seed reporting. In it, `pqk` is ahead of logit KD in 3 of 6 pairs and ahead of its twin in 2 of 6 at its pre-specified scale. (ii) Two ablations that probe where a quantum-kernel effect could come from. Removing the circuit's entanglement lowers TrashNet validation macro-F1 on every seed but raises it on RPS-25, and a teacher projection fitted to the labels beats the random one in only one of eight arm-pairs. (iii) Deployment evidence on a Raspberry Pi 4B. The INT8 students run in 2.5 and 3.2 ms, and the device's default kernels change 0.3% to 9.4% of a model's predictions relative to the PC. Section II reviews related work, Section III describes the method, Section IV the setup, and Section V the results.

II. Related Work

a) Knowledge distillation: Logit distillation trains a student on a teacher's softened outputs [5]. Relational KD (RKD) instead aligns pairwise distances and angles of the embeddings [6], and contrastive distillation pursues the same idea with a different objective [7]. A trained network's predictions are determined by inner products of its last-layer features, and this feature kernel is a natural object to distill [19]. Our kernel-alignment loss belongs to this family. It matches batch Gram matrices of teacher and student after projecting both to eight angles, and its classical twin replaces only the kernel.

b) Quantum kernels and their limits: The projected quantum kernel [16] compares reduced states of an embedded system instead of the full state. Whether quantum kernels help on classical data is contested. Kernel values can concentrate exponentially, and entanglement, noise and expressivity are among the identified sources [13]. Tuned fidelity kernels show no advantage on classical data [14]. An advantage can also vanish under noise and finite data [20], and bandwidth-tuned quantum kernels have been argued to behave classically [15]. What a quantum kernel can learn depends on the inductive bias of its embedding [21], and a large benchmark studies which design choices make such kernels effective [22]. These results shape our design. We pair the quantum kernel with a classical RBF kernel on the same inputs, and one quantum variant receives the same median-heuristic scale as its twin. The comparison can then separate the kernel from its calibration.

c) Quantum-assisted distillation: QRKD [9] extends RKD with quantum-kernel values. It evaluates CNNs on MNIST and CIFAR-10 and GPT-2 on three language benchmarks, and reports consistent gains over classical RKD while all deployed models stay classical. Classical teachers have also been distilled into quantum students [10, 11]. A variational circuit has served as a soft-label teacher for a classical student [12], and quantum-inspired compression has been combined with distillation for resource-constrained deployment [23]. Our setting is closest to QRKD. It differs in the data (small, imbalanced waste images), the students, the loss, the controls (a classical twin and two ablations), the reporting (paired seeds) and the end point (INT8 models on a Raspberry Pi). We do not claim that those results fail in their own settings.

d) Edge deployment: Integer-arithmetic-only inference [4] underlies the 8-bit deployment used here, and we apply post-training quantization to finished students. Edge-oriented studies examine accuracy–efficiency trade-offs of distillation for CNNs and vision transformers [24], evaluate distillation and compression on edge-class devices [8], and split distilled networks between device and server [25]. We measure a single compact INT8 model on the device itself and report how its predictions depend on the kernels that execute it.

III. Method

Let x be an image with label y. We write $z_t(x) \in \mathbb{R}^{2048}$ and $z_s(x) \in \mathbb{R}^{d_s}$ for the penultimate features of the teacher and the student, and ℓ_t, ℓ_s for their logits. All losses are computed on batches of up to $B = 128$ images. The last batch of an epoch is smaller, and the 33-image RPS-25 validation split forms a single batch.

A. Baseline objectives

`scratch` minimizes the cross-entropy $\mathrm{CE}(\ell_s, y)$. `kd` is Hinton distillation [5],

$$L_{\mathrm{KD}} = \alpha\,\mathrm{CE}(\ell_s, y) + \beta\,\tau^2\,\mathrm{KL}(\mathrm{softmax}(\ell_t/\tau)\,\|\,\mathrm{softmax}(\ell_s/\tau))\,, \tag{1}$$

with $\alpha = \beta = 0.5$. The temperature τ was screened on some pairs and reused on others (Section IV-B). `rkd` [6] adds distance-wise and angle-wise relational terms on the penultimate features,

$$L_{\mathrm{RKD}} = \mathrm{CE}(\ell_s, y) + w_d\,\mathrm{SmoothL1}\big(\psi_D(z_s), \psi_D(z_t)\big) + w_a\,\mathrm{SmoothL1}\big(\psi_A(z_s), \psi_A(z_t)\big), \tag{2}$$

where ψ_D collects the pairwise distances in a batch, normalized by their mean, and ψ_A the cosines of the angles formed by triplets. Both potentials are dimension-free, so a 2048-dimensional teacher can be compared directly with a student of any width.

B. Kernel alignment

The two kernel methods, `pqk` and its twin `rbf_control`, add a kernel-alignment term to the logit objective:

$$L = \alpha\,\mathrm{CE}(\ell_s, y) + \beta\,\tau^2\,\mathrm{KL}(\cdot) + \gamma_e\,L_{\mathrm{kernel}}, \tag{3}$$

$$L_{\mathrm{kernel}} = \big\|\tilde{K}_t - \tilde{K}_s\big\|_F^2, \qquad \tilde{K} = \frac{K}{\mathrm{tr}\,K}. \tag{4}$$

Here $\alpha = \beta = 0.5$, and all kernel methods share the temperature $\tau = 0.5$. The weight $\gamma_e = \gamma_{\max}\min\big(1, (e{+}1)/10\big)$ rises linearly over the first ten epochs e to $\gamma_{\max} = 1$. Validation is always scored at $\gamma_{\max}$, so the ramp cannot change which checkpoint

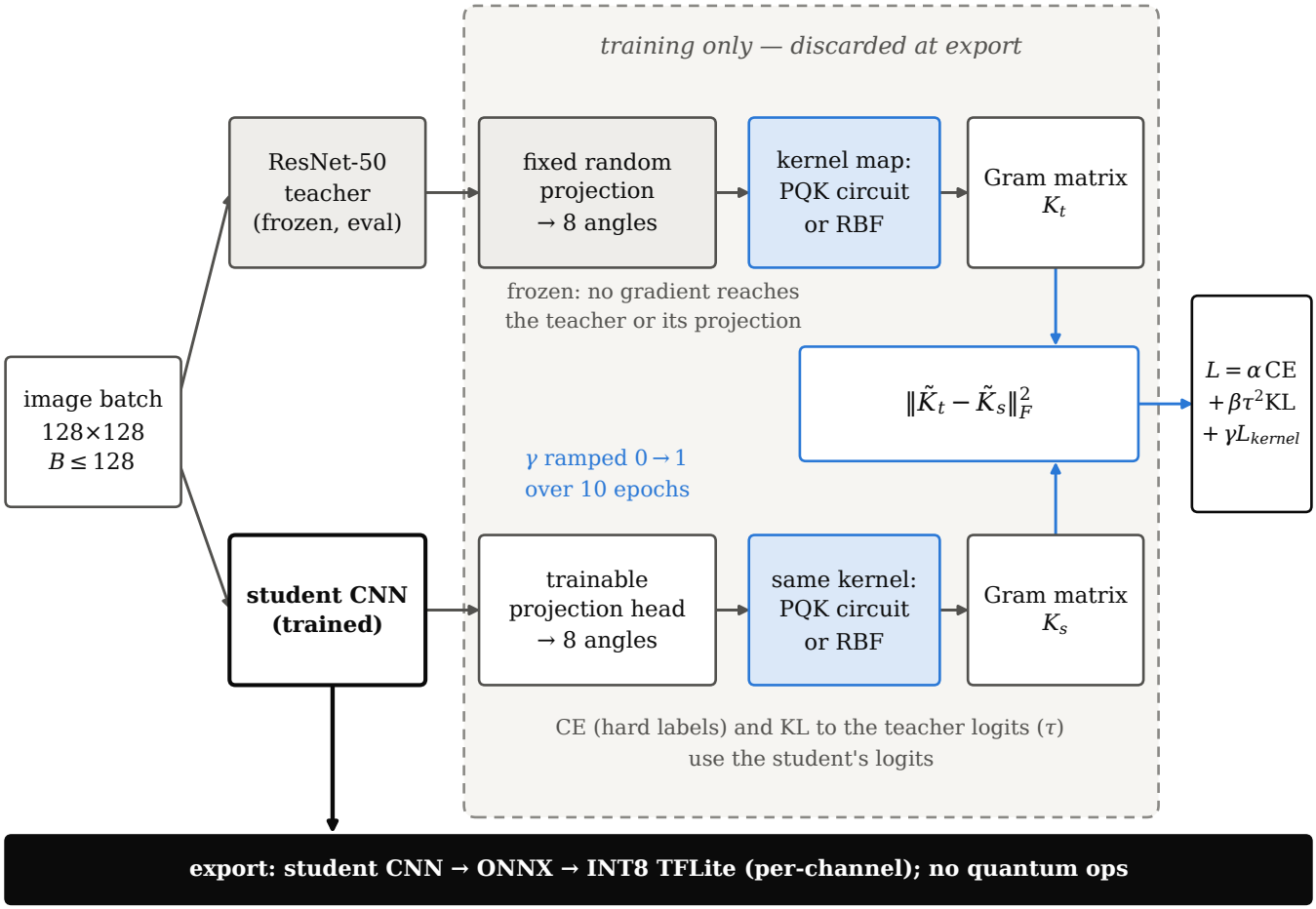


Fig. 1. Training-time pipeline. Penultimate features of the frozen teacher and of the student are projected to eight angles and compared through the Gram matrices of one kernel (the PQK circuit or an RBF kernel). Everything inside the dashed region is discarded at export; only the student CNN is deployed.

is selected. K_t and K_s are the $B \times B$ Gram matrices of the teacher and the student, built from eight angles per image:

$$\theta_s = \pi\,\sigma(W_s z_s + b_s), \quad \theta_t = \pi\,\sigma(W_t z_t + b_t), \quad \theta \in [0,\pi]^8. \tag{5}$$

The student-side projection (W_s, b_s) is trained jointly with the student, and gradients reach the student through the kernel. The teacher-side projection (W_t, b_t) is drawn at default initialization at the start of each run, then frozen and evaluated without gradients. Freezing it is necessary. If both projections were trainable, collapsing both to constant maps would set every Gram entry to one and make the two normalized matrices identical, so L_{kernel} would reach zero without the student learning anything. Our first implementation had this flaw: in the first `pqk` run we inspected (RPS-25, LeNet-5, seed 0), the mean off-diagonal kernel value was 0.995 for the student and 0.986 for the teacher. We now log the mean off-diagonal kernel value in every epoch and treat values below 0.05 or above 0.95 as degenerate.

C. The projected quantum kernel

The quantum kernel is the projected quantum kernel of Huang et al. [16]. The circuit (Figure 2) acts on 8 qubits that start in $|0\rangle^{\otimes 8}$. It applies $R_Y(\theta_q)$ to every qubit q, then a CNOT from qubit q to $q{+}1$ for $q = 0, \ldots, 6$, and repeats this pair of layers 4 times. The CNOTs form an open chain; no gate closes it into a ring. For each input we measure the Bloch vector of every qubit, $r(\theta) = (\langle X_q\rangle, \langle Y_q\rangle, \langle Z_q\rangle)_{q=0}^{7}$. R_Y and CNOT are real matrices, so the state stays real and $\langle Y_q\rangle \equiv 0$. Only 16 of the 24 coordinates are therefore active. The kernel compares inputs through their single-qubit reduced states $\rho_q = \frac{1}{2}(I + r_q \cdot \sigma)$,

$$\begin{aligned} K(\theta,\theta') &= \exp\Big(-\lambda \sum_q \big\|\rho_q(\theta) - \rho_q(\theta')\big\|_F^2\Big) \\ &= \exp\Big(-\tfrac{\lambda}{2}\,\big\|r(\theta) - r(\theta')\big\|^2\Big), \end{aligned} \tag{6}$$

where the second form follows from $\mathrm{tr}(\sigma_a\sigma_b) = 2\delta_{ab}$. Only single-qubit expectation values are read out, and at eight qubits the circuit is cheap to simulate exactly. We use PennyLane [26] with its state-vector `lightning.qubit` backend and adjoint differentiation.

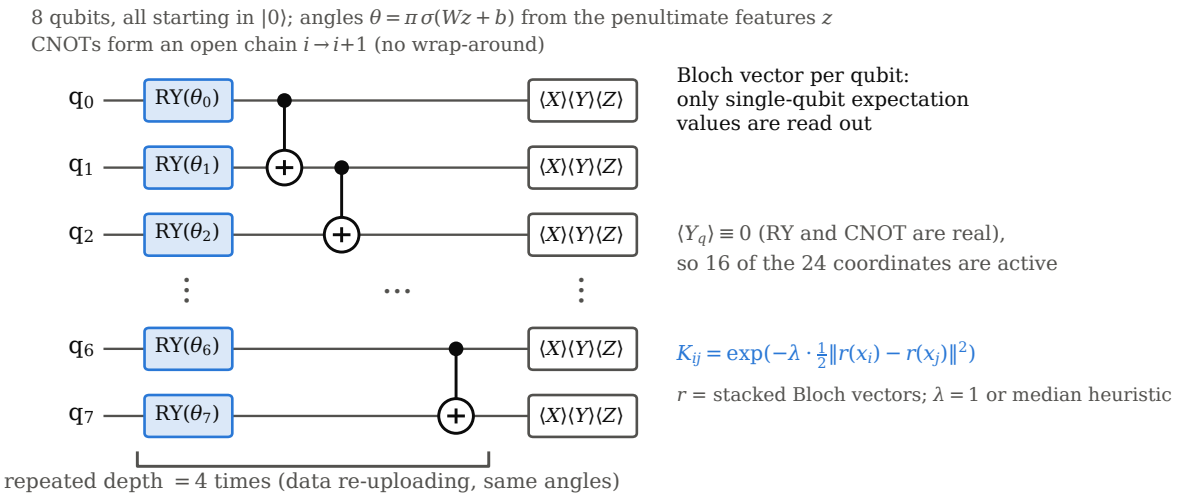


Fig. 2. The fixed feature map behind `pqk` (shown for qubits 0, 1, 2 and 7). Each of the eight angles drives an R_Y rotation, followed by an open chain of CNOT gates; the pair is repeated 4 times with the same angles. Only single-qubit expectation values are read out. Nothing in the circuit is trainable.

We set the scale λ in two ways. `pqk` uses the pre-specified $\lambda = 1$. The variant `pqk_adaptive`, adopted after the first results (Section IV-C), sets λ in every batch so that $K = \exp\big(-d^2/(2m)\big)$, where $d^2 = \frac{1}{2}\|\Delta r\|^2$ and m is the median off-diagonal d^2 of the batch. The Gram matrix is then the median-heuristic RBF kernel on the measured features $r(\theta)$.

D. Matched classical controls

`rbf_control` is the same pipeline with an RBF kernel, $K = \exp\big(-\|\theta - \theta'\|^2/(2m)\big)$, applied to the same eight angles with the same median heuristic. The quantum branch with adaptive λ and `rbf_control` thus differ only in the map $\theta \mapsto r(\theta)$ that precedes an identically calibrated RBF kernel. Two further arms isolate where an effect of that map could come from.

- `noent_control` deletes the CNOT gates. Each qubit then sees $R_Y(\theta_q)^4 = R_Y(4\theta_q)$, so $r = (\sin 4\theta_q, 0, \cos 4\theta_q)$ exactly (checked against the simulator to 6×10^{-8}). The map keeps the 16 active coordinates, their range, the absence of trainable parameters and the adaptive RBF kernel, and differs from `pqk_adaptive` only by entanglement.
- `pqk_fit` and `rbf_fit` replace the random teacher-side projection with a map fitted once on the training split and then frozen. It consists of the $C-1$ shrinkage-LDA directions, completed to eight outputs with the leading principal components of the residual and standardized on the training set.

A third control, random Fourier features drawn from the circuit's own frequency spectrum, was implemented but not run. All kernel arms use eight angles, the loss of Eq. (3) and the same frozen teacher, and every circuit has 8 qubits and depth 4.

E. What is trained and what is deployed

Only the student CNN is exported. The projection heads, the circuit and the kernels exist only inside the training loop, and a purity check fails if any of them appears in the exported graph (Section IV-D). For every method, including `pqk`, the deployed model is an ordinary convolutional network; the quantum kernel affects it only through the weights it learned.

IV. Experimental Setup

A. Data

TrashNet [1] has 2,527 photographs in six classes (cardboard, glass, metal, paper, plastic, trash) and no official split. We split it once, stratified 70/15/15 with seed 0 (1,768/379/380 images), and never re-split it. The classes are imbalanced (96 to 416 training images per class). Macro-F1 is therefore the primary metric, and training batches use a class-balanced sampler. RPS-25 is a 25% subset of the Rock–Paper–Scissors training images [17] (630 images, 210 per class). We kept it deliberately low-data so that distillation has room to help. It keeps the official validation (33 images) and test (372 images) splits, and top-1 accuracy is its primary metric. We treat it as a low-data confirmation, not as a second application. All images are resized to 128×128 and normalized with ImageNet statistics. Training uses random resized crops, RandAugment [27] and horizontal flips, identically for every method. The crop's minimum scale is 0.85 on TrashNet and 0.7 on RPS-25, and RandAugment applies one operation at magnitude 5 on TrashNet and two at magnitude 9 on RPS-25.

B. Models and training

The teacher is an ImageNet-pretrained ResNet-50 [18], fine-tuned once per dataset (AdamW, learning rate 10^{-4}, at most 20 epochs, patience 5) and then frozen in evaluation mode. Its test macro-F1 is 0.876 on TrashNet and 0.907 on RPS-25. The students are trained from random initialization. They are MobileNetV2 with width multiplier 0.35 (0.40 M parameters), MobileNetV3-Small (0.93 M) and a LeNet-5 variant with global average pooling before the classifier (0.03 M). Only the two MobileNets, the edge-relevant students, are exported.

All methods on a pair share the data, the optimizer (AdamW, learning rate 10^{-3}, weight decay 10^{-4}, cosine schedule), the batch size (128), the epoch budget and the early-stopping patience. The budget is 60 epochs with patience 10 for MobileNetV2-0.35, 40 and 8 for MobileNetV3-Small, and 60 and 12 for LeNet-5. The best validation checkpoint is restored. On TrashNet it is selected by validation macro-F1. On RPS-25, whose 33-image validation split makes F1 too erratic, it is selected by validation loss, meaning each method's own objective with the kernel term at $\gamma_{\max}$. On RPS-25, and for MobileNetV3-Small on TrashNet, the running BatchNorm statistics did not stabilize. There, BatchNorm uses per-batch statistics during training, and a recalibration pass on clean training images precedes every evaluation. `kd` uses $\alpha = \beta = 0.5$ with $\tau = 3$ on TrashNet and $\tau = 1$ on RPS-25. Both values were screened on MobileNetV2-0.35 and reused for the other students. `rkd` uses distance and angle weights 5 and 10 (1 and 2 for LeNet-5 on TrashNet). The kernel methods use $\tau = 0.5$ and the γ ramp of Section III-B. Two cells depart from the shared recipe, both on RPS-25/LeNet-5. One `scratch` seed used a 40-epoch budget with patience 8, and the `kd` cell mixes $\tau = 0.5$ (seed 0) and $\tau = 1$ (seeds 1 and 2). We did not re-run them; Section V states where they matter. Each cell is trained with seeds 0, 1 and 2, which set the student initialization, the data order and the random teacher projection.

C. Selection, reporting and the status of the results

Every number is the mean ± sample standard deviation ($n-1$) over three seeds. Comparisons are paired by seed. We report the mean paired difference, its standard deviation and the number of seeds on which the first method is ahead. Two-sided paired t-tests are descriptive and uncorrected for multiple comparisons. For the follow-up arms, a shared seed gives identical student initialization and data order, because the first-batch cross-entropy and KL gradients are identical across the arms. Stored logs confirm this for `noent_control`, `pqk_fit` and `rbf_fit` in every cell, and for two `pqk_adaptive` runs. For the earlier grid, pairing means a shared seed value only.

The results are exploratory, for three reasons. First, the student recipes, the KD temperature and the RKD weights were screened on seed 0, and some of these choices were judged by test scores. Second, the kernel methods' $\tau = 0.5$ was inherited from a KD screen on LeNet-5 with RPS-25 and was not re-tuned, whereas the `kd` temperature was screened. Comparisons between a kernel method and `kd` are therefore not matched on that parameter and may disadvantage the kernel methods. Third, the scale λ of `pqk` was chosen after results were seen. The pre-specified $\lambda = 1$ gave the `pqk` results of the first full grid. The adaptive λ was adopted afterwards, per pair, by the higher three-seed *validation* macro-F1 of the two variants. For this choice and for the follow-up analyses, validation decides and test scores are descriptive. The headline comparisons show both λ variants. A confirmatory claim would need a frozen protocol, selection on validation alone and fresh data.

D. INT8 export and on-device benchmark

Each MobileNet student is stripped of its projection and kernel modules and converted to ONNX (opset 17, static batch size 1). It is then converted to a fully integer TFLite model with per-channel weight quantization, calibrated on 256 unaugmented training images. Every model passes four checks: purity (no projection, quantum or custom operations), a fully integer, per-channel graph, ONNX–PyTorch logit parity, and an fp32 reload that reproduces the recorded test score. INT8 test scores are measured with the LiteRT interpreter on an x86 PC ("PC" below). The 66 exported models cover two students, two datasets, five methods and three seeds, plus the second λ variant of `pqk` on two pairs. For TrashNet/MobileNetV2-0.35 the exported `pqk` uses $\lambda = 1$, whereas the final validation rule prefers adaptive λ.

The INT8 models are benchmarked on a Raspberry Pi 4 Model B (Cortex-A72, four cores, 4 GB, Ubuntu 20.04, 600–1800 MHz) with LiteRT (`ai-edge-litert` 2.1.0). Each model runs in a fresh process with 50 warm-up and 500 timed invocations at batch size 1, starting below 55 °C. We log latency percentiles, CPU use, memory, SoC temperature and clock, and the firmware's throttling flags. The suite runs every model with the default XNNPACK delegate and with the builtin kernels. It adds a thread sweep from one to four threads, fp32 references, on-device test scores compared with the PC's predictions, and a live USB-camera pipeline (640×480). Energy, the Jetson Orin Nano and the ESP32-S3 were not measured.

All training ran on a single GPU, often with several runs sharing it, so we report compute-hours rather than per-epoch times. The 152 student runs include the grid, the follow-up arms, the screens, and seven runs of two wider MobileNetV2 variants that were stopped as capacity probes and are not reported.

V. Results

A. Distillation baselines

Table I and Figure 3 show that distillation helps in some pairs but not in others. `kd` beats `scratch` on TrashNet for both MobileNets, by +0.025 ± 0.008 (MobileNetV2-0.35) and +0.009 ± 0.007 (MobileNetV3-Small) macro-F1, and it is ahead on all three seeds of each. On LeNet-5 it hurts (−0.045 ± 0.093), while `rkd` helps (+0.049 ± 0.035). The best mean is `kd` on the TrashNet MobileNets, `rkd` on TrashNet/LeNet-5 and RPS-25/MobileNetV2-0.35, `rbf_control` on RPS-25/MobileNetV3-Small, and `scratch` on RPS-25/LeNet-5.

TABLE I

TEST MACRO-F1 AND TOP-1 ACCURACY, MEAN ± SAMPLE STD OVER THREE SEEDS. BOLD: BEST MEAN IN A COLUMN WITHIN A STUDENT. †PQK WITH THE VALIDATION-SELECTED ADAPTIVE λ (ELSEWHERE $\lambda = 1$). §ONE SEED USED A DIFFERENT TRAINING BUDGET; ¶THE SEEDS OF THIS KD CELL USED TWO TEMPERATURES (SECTION IV-B). RKD WAS NOT PART OF THE FINAL GRID FOR RPS-25/LENET-5. THE PRIMARY METRIC IS MACRO-F1 ON TRASHNET AND TOP-1 ON RPS-25.

		TrashNet (6 classes)		RPS-25 (3 classes)	
Student	Method	Macro-F1	Top-1	Macro-F1	Top-1
MobileNetV2-0.35	scratch	0.684 ± 0.014	0.697 ± 0.021	0.710 ± 0.023	0.727 ± 0.030
	kd	**0.708** ± 0.013	**0.725** ± 0.009	0.710 ± 0.088	0.719 ± 0.087
	rkd	0.672 ± 0.004	0.686 ± 0.003	**0.737** ± 0.051	**0.745** ± 0.048
	rbf_control	0.671 ± 0.010	0.692 ± 0.012	0.684 ± 0.062	0.702 ± 0.059
	pqk	0.683 ± 0.013†	0.694 ± 0.016†	0.713 ± 0.030†	0.730 ± 0.019†
MobileNetV3-S	scratch	0.724 ± 0.016	0.745 ± 0.016	0.816 ± 0.053	0.828 ± 0.045
	kd	**0.732** ± 0.020	**0.753** ± 0.021	0.830 ± 0.089	0.843 ± 0.075
	rkd	0.716 ± 0.003	0.736 ± 0.007	0.849 ± 0.048	0.859 ± 0.041
	rbf_control	0.715 ± 0.037	0.730 ± 0.040	**0.888** ± 0.063	**0.892** ± 0.060
	pqk	0.718 ± 0.009†	0.735 ± 0.008†	0.840 ± 0.065	0.851 ± 0.054
LeNet-5	scratch	0.529 ± 0.021	0.541 ± 0.022	**0.676** ± 0.108§	**0.676** ± 0.109§
	kd	0.484 ± 0.074	0.490 ± 0.075	0.650 ± 0.012¶	0.652 ± 0.016¶
	rkd	**0.578** ± 0.016	**0.597** ± 0.021	–	–
	rbf_control	0.539 ± 0.008	0.552 ± 0.007	0.538 ± 0.202	0.584 ± 0.153
	pqk	0.535 ± 0.003	0.552 ± 0.003	0.638 ± 0.070	0.642 ± 0.058

pqk is best on no pair. rbf_control beats scratch on the mean in only 2 of 6 pairs. Its largest gain is on RPS-25/MobileNetV3-Small (+0.072 ± 0.034), where it is ahead on all three seeds. On TrashNet all students stay well below the teacher (best 0.732 against 0.876 macro-F1). On RPS-25 the best student (0.888) comes close to the teacher's 0.907. Seed noise is large relative to these effects. The standard deviation across seeds ranges from 0.012 to 0.202 on RPS-25 and from 0.003 to 0.074 on TrashNet.

B. PQK against logit KD, the RBF twin and relational KD

a) Against logit KD: pqk and kd are not distinguishable (Table II). At fp32, pqk is ahead on the mean in 3 of 6 pairs, or in 4 if the one kd seed trained at a different temperature is excluded. On TrashNet, the primary dataset, it is behind on both MobileNets, by 0.025 ± 0.025 and 0.014 ± 0.021 macro-F1, and ahead on one seed of three in each. It is ahead on TrashNet/LeNet-5 (+0.051 ± 0.073), where kd itself fails. On RPS-25 the differences are +0.003, +0.010 and −0.013 on macro-F1, and +0.012, +0.008 and −0.010 on top-1, the primary metric there. After INT8 conversion, pqk is ahead in 3 of 4 pairs on PC numerics and on the Pi's builtin kernels, and in 2 of 4 with the Pi's default XNNPACK kernels. TrashNet/MobileNetV3-Small is a loss under all three (−0.017, −0.016, −0.025), with pqk behind on every seed. The RPS-25 differences (+0.011 to +0.032) come with paired standard deviations of 0.10 to 0.12. No difference is significant; the smallest uncorrected p-value is 0.22 at fp32 and 0.10 at INT8. The comparison is not matched on temperature, because the kd temperature was screened while the kernel methods use one untuned τ. Against scratch, pqk differs by −0.039 to +0.024 across the six pairs (smallest $p = 0.37$).

b) Against the RBF twin: At the pre-specified $\lambda = 1$, pqk is ahead of its twin on the mean in only 2 of 6 pairs (Table III). These are TrashNet/MobileNetV2-0.35 (+0.023 ± 0.016, three of three seeds) and RPS-25/LeNet-5 (+0.100 ± 0.145, two of three). It is behind on the other four (−0.042, −0.004, −0.028, −0.048). The two winning pairs were the first to be evaluated, and the other four did not reproduce their result. The adaptive λ gives pqk the same calibration as the twin. With it, pqk is ahead on 3 of 4 pairs (+0.013 ± 0.011, +0.003 ± 0.036, +0.029 ± 0.088) and behind on RPS-25/MobileNetV3-Small (−0.058 ± 0.011), winning on 3, 2, 1 and 0 seeds respectively. The smallest p-value is 0.13 for $\lambda = 1$. For the adaptive variant it is 0.01, and that difference favours the classical twin. The quantum kernel therefore shows no consistent advantage over its classical counterpart.

c) Against relational KD: rkd is the classical counterpart of QRKD's quantum extension. pqk is ahead of rkd on the two TrashNet MobileNet pairs (+0.012 ± 0.016 and +0.003 ± 0.008). It is behind on TrashNet/LeNet-5 (−0.043 ± 0.018, on every seed, $p = 0.052$), RPS-25/MobileNetV2-0.35 (−0.024 ± 0.023) and RPS-25/MobileNetV3-Small (−0.009 ± 0.111). Our setting differs from that of QRKD, so this is not a failed replication. We do not, however, observe consistent gains over classical relational distillation.

C. Seed-to-seed variance

The one pattern in favour of pqk is a lower spread across seeds (Table IV), and it is weaker than it first looks. On macro-F1, with the validation-selected λ, its standard deviation is lower than that of rbf_control on 4 of 6 pairs and lower than that of scratch on 4. On top-1 the counts are 5 and 5. Pooled over pairs, the standard deviation is 0.041 for pqk, against 0.092 for rbf_control and 0.052 for scratch. The scratch value includes the recipe-mismatched run on RPS-25/LeNet-5; without that seed it falls to 0.032, below that of pqk. The advantage over rbf_control is therefore the more consistent one, and the advantage over scratch is fragile. Three further limits apply. A standard deviation from

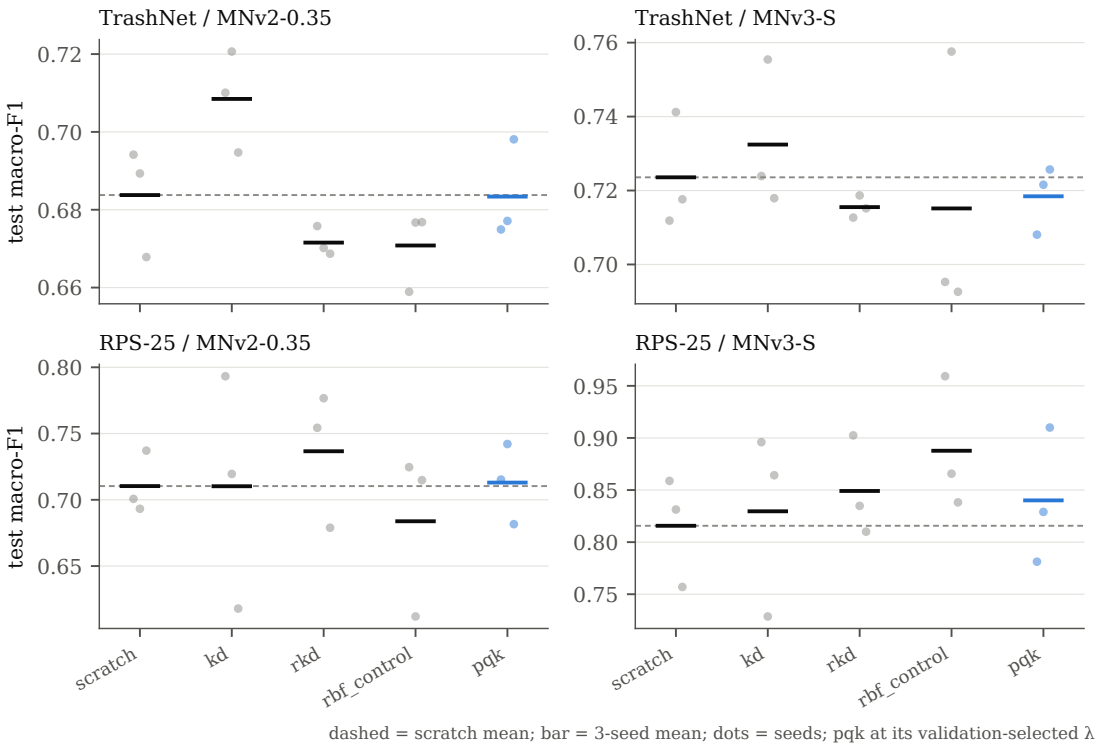


Fig. 3. Test macro-F1 of the five methods for the four MobileNet pairs. Dots are individual seeds, bars are means, and the dashed line is the mean of `scratch`.

TABLE II

PAIRED DIFFERENCE `pqk` − `kd` IN TEST MACRO-F1 (MEAN ± STD OF THE PER-SEED DIFFERENCES, AND THE SEEDS ON WHICH `PQK` IS AHEAD). `PQK` IS AT ITS VALIDATION-SELECTED λ AT FP32 AND AT THE EXPORTED VARIANT AT INT8; THESE DIFFER ONLY FOR TRASHNET/MOBILENETV2-0.35 ($\lambda = 1$ AT INT8). "PC" USES LITERT'S BUILTIN KERNELS ON X86; THE "PI" COLUMNS WERE MEASURED ON THE RASPBERRY PI WITH ITS BUILTIN KERNELS AND WITH ITS DEFAULT XNNPACK DELEGATE. ¶SEE TABLE I. LENET-5 WAS NOT EXPORTED.

		fp32	INT8		
Dataset	Student	test macro-F1	PC (builtin)	Pi builtin	Pi XNNPACK
TrashNet	MobileNetV2-0.35	−0.025 ± 0.025 (1/3)	+0.004 ± 0.006 (2/3)	+0.004 ± 0.006 (2/3)	−0.004 ± 0.010 (1/3)
TrashNet	MobileNetV3-S	−0.014 ± 0.021 (1/3)	−0.017 ± 0.021 (0/3)	−0.016 ± 0.016 (0/3)	−0.025 ± 0.015 (0/3)
TrashNet	LeNet-5	+0.051 ± 0.073 (2/3)	–	–	–
RPS-25	MobileNetV2-0.35	+0.003 ± 0.073 (2/3)	+0.025 ± 0.108 (2/3)	+0.025 ± 0.108 (2/3)	+0.032 ± 0.113 (2/3)
RPS-25	MobileNetV3-S	+0.010 ± 0.112 (2/3)	+0.016 ± 0.101 (2/3)	+0.017 ± 0.104 (2/3)	+0.011 ± 0.115 (2/3)
RPS-25	LeNet-5	−0.013 ± 0.059 (2/3)¶	–	–	–
Pairs where `pqk` is ahead on the mean		3/6	3/4	3/4	2/4

three seeds is itself very uncertain, and a sign test on four of six pairs gives $p = 0.69$. The pattern depends on the calibration: with $\lambda = 1$, `pqk` is below `scratch` on three pairs and below `rbf_control` on five. Without the mismatched seed, the count against `scratch` is 3 of 6 on macro-F1 and 4 on top-1. `pqk` is also not the most stable method. On the five pairs where `rkd` exists, the pooled standard deviations are 0.029 for `scratch`, 0.032 for `rkd` and 0.033 for `pqk`. After INT8 conversion the pattern weakens further. Pooled over the four exported pairs on PC numerics, `pqk` has 0.042, against 0.037 for `scratch` and 0.025 for `rbf_control`. On TrashNet, however, it remains below both on both students under all three sets of kernels.

D. Entanglement and the teacher projection

a) Removing the entanglement: On TrashNet, whose validation split has 379 images, `pqk` (adaptive λ) is ahead of its entanglement-free twin on every seed of both students, with validation macro-F1 differences of $+0.049 \pm 0.065$ and $+0.007 \pm 0.008$. This is the one consistent sign that the CNOT gates contribute something beyond the sixteen trigonometric coordinates. On RPS-25, with a 33-image validation set, the sign reverses (-0.041 and -0.022). Test scores point the same way, except on TrashNet/MobileNetV3-Small, where the difference vanishes (-0.002). The claim is limited. The entanglement-free readout $(\sin 4\theta, \cos 4\theta)$ oscillates rapidly over $\theta \in (0, \pi)$ and may simply be a poor classical map,

TABLE III

TEST MACRO-F1 OF THE TWIN RBF_CONTROL AND THE PAIRED DIFFERENCE pqk − rbf_control FOR THE TWO λ CALIBRATIONS OF PQK. ADAPTIVE λ HAS ONE SEED ON THE LENET-5 PAIRS AND IS OMITTED THERE. [§] SEE TABLE I.

Dataset	Student	scratch	rbf_control	Δ pqk (λ=1)	Δ pqk (adaptive λ)	selected
TrashNet	MobileNetV2-0.35	0.684 ± 0.014	0.671 ± 0.010	+0.023 ± 0.016 (3/3)	+0.013 ± 0.011 (3/3)	adaptive
TrashNet	MobileNetV3-S	0.724 ± 0.016	0.715 ± 0.037	−0.042 ± 0.049 (1/3)	+0.003 ± 0.036 (2/3)	adaptive
TrashNet	LeNet-5	0.529 ± 0.021	0.539 ± 0.008	−0.004 ± 0.010 (2/3)	–	λ=1
RPS-25	MobileNetV2-0.35	0.710 ± 0.023	0.684 ± 0.062	−0.028 ± 0.082 (1/3)	+0.029 ± 0.088 (1/3)	adaptive
RPS-25	MobileNetV3-S	0.816 ± 0.053	0.888 ± 0.063	−0.048 ± 0.125 (1/3)	−0.058 ± 0.011 (0/3)	λ=1
RPS-25	LeNet-5	0.676 ± 0.108[§]	0.538 ± 0.202	+0.100 ± 0.145 (2/3)	–	λ=1
Pairs where pqk beats its twin on the 3-seed mean					2/6 (λ=1); 3/4 (adaptive)	

TABLE IV

STANDARD DEVIATION OF TEST MACRO-F1 ACROSS THE THREE SEEDS. BOLD: LOWEST OF THE FIVE METHODS, WITH PQK AT ITS VALIDATION-SELECTED λ (LAST COLUMN); THE FIXED AND ADAPTIVE COLUMNS SHOW BOTH CALIBRATIONS. THE LAST ROW IS THE ROOT MEAN SQUARE OVER PAIRS (ADAPTIVE: FOUR PAIRS, RKD: FIVE). [§],[¶] SEE TABLE I.

Dataset	Student	scratch	kd	rkd	rbf_control	pqk λ=1	pqk adaptive	pqk selected
TrashNet	MobileNetV2-0.35	0.014	0.013	**0.004**	0.010	0.006	0.013	0.013
TrashNet	MobileNetV3-S	0.016	0.020	**0.003**	0.037	0.023	0.009	0.009
TrashNet	LeNet-5	0.021	0.074	0.016	0.008	0.003	–	**0.003**
RPS-25	MobileNetV2-0.35	**0.023**	0.088	0.051	0.062	0.031	0.030	0.030
RPS-25	MobileNetV3-S	0.053	0.089	**0.048**	0.063	0.065	0.074	0.065
RPS-25	LeNet-5	0.108[§]	**0.012**[¶]	–	0.202	0.070	–	0.070
Pooled std (root-mean-square over pairs)		0.052	0.060	0.032	0.092	0.042	0.041	0.041

and the generic control designed to settle this was not run. The class separation of the target kernel did not predict the outcome either. On TrashNet the entanglement-free target is better aligned with the labels (0.323 against 0.245 centered kernel–target alignment on validation), yet students trained on the PQK target rank higher.

b) A fitted teacher projection: The fixed random 2048 → 8 projection discards much of the teacher's class structure. On TrashNet validation, the alignment of the teacher kernel with the label kernel falls from 0.720 for the raw features to 0.363 for the random angles. A projection fitted on the training split (LDA+PCA, selected before any student was trained) restores it to 0.825. It did not help: fitted was better than random in one of eight arm-pairs on validation, the exception being pqk on RPS-25/MobileNetV3-Small (+0.043 ± 0.039). On TrashNet the fitted target is almost the label kernel on the training batches where the loss is computed. Its alignment is 0.963 for the RBF target against 0.825 on validation, and 0.902 against 0.752 for the PQK target. Aligning to such a target asks the student to cluster by class, which cross-entropy already does, and removes the within-class geometry that relational distillation is meant to transfer. This account does not explain RPS-25, where the fitted PQK target (0.897 on training batches against 0.797 on validation) produced the one positive case, and we did not test it by intervention.

c) Gradient balance: With the random target, the kernel term does not dominate training. The ratio of its gradient norm to that of the cross-entropy falls from 0.70 over the first three epochs to 0.14 over the last five for the entanglement-free arm (12 runs), and from 0.88 to 0.20 for pqk (two logged runs). With a fitted target the ratio stays between 0.28 and 0.74 on three pairs. On RPS-25/MobileNetV2-0.35 it reaches 3.1 and 2.1, so there the kernel term outweighs the label signal.

E. INT8 conversion and on-device behaviour

All 66 exported models pass every check. They are fully integer and per-channel, contain no custom or floating-point operations, and reproduce their fp32 score on reload; their ONNX graphs match the PyTorch logits to within 1.3×10^{-4}. Model size is set by the architecture alone, 0.62 MB for MobileNetV2-0.35 and 1.21 MB for MobileNetV3-Small, so any comparison at matched size holds by construction. Conversion has no systematic cost. Macro-F1 changes by +0.003 ± 0.022 across the models, and 37 of them improve. Conversion does change predictions: INT8 and fp32 disagree on about 7% of test images, in both directions. INT8 rankings between methods are therefore fp32 rankings plus noise. pqk has the most favourable fp32-to-INT8 change on PC numerics (+0.0080, up on 9 of 12 models, against −0.0031 for kd). The difference is not significant ($p = 0.22$), and the change shrinks to +0.0025 with the Pi's default kernels.

On the Raspberry Pi (160 runs, none failed; Table V), the median INT8 latency with four threads is 2.5 ms for MobileNetV2-0.35 and 3.2 ms for MobileNetV3-Small, about 395 and 315 inferences per second. Latency is a property of the architecture: within a pair, the spread between methods is at most 0.11 ms. The two networks do about the same work (18.9 and 18.2 M multiply–accumulates). Even so, MobileNetV2 is about 25% faster on four threads and scales better with threads (1.75–1.84× against 1.35–1.40×). Against fp32 with the same runtime, INT8 is 1.2 to 1.4× faster and 2.6 to 3.1× smaller, and it uses 5.4–6.7 MB of interpreter memory instead of 8–12 MB.

TABLE V

INT8 models on the Raspberry Pi 4B with the default XNNPACK delegate, four threads, batch size 1: median (p50) and 95th-percentile (p95) invocation time over 500 calls, mean throughput and the speed-up from one to four threads. Latency and throughput are averages over the methods and seeds of each pair (15–18 models); the thread speed-up is measured on one model per pair. Sizes are in KiB.

Dataset	Student	Params	INT8 KiB	MMACs	p50 ms	p95 ms	FPS	1→4 thr.
TrashNet	MobileNetV2-0.35	403,814	608	18.9	2.53	2.71	395	1.84×
TrashNet	MobileNetV3-S	930,470	1181	18.2	3.18	3.35	314	1.35×
RPS-25	MobileNetV2-0.35	399,971	604	18.9	2.52	2.73	395	1.75×
RPS-25	MobileNetV3-S	928,739	1179	18.2	3.14	3.30	318	1.40×

XNNPACK is 1.1 to 1.4× faster than the builtin kernels, but it changes predictions. Per model, it disagrees with the PC on 0.3% to 9.4% of test images (3.5% on average), against at most 2.4% for the builtin kernels. These disagreements move per-method macro-F1 by -0.016 to +0.012. They also flip the sign of the `pqk−kd` comparison on TrashNet/MobileNetV2-0.35 (+0.004 on PC, −0.004 with XNNPACK), so INT8 results should be reported with the runtime that produced them. The SoC reached at most 64.8 °C. The firmware did flag under-voltage and throttling in 2 of the 160 runs, so the supply is marginal and we attempted no energy measurement. A live-camera pipeline reaches the webcam's 30 frames per second with two inference and two OpenCV threads, at 12.8 ms of processing per frame. Image preparation (8.8 ms) takes longer than inference (4.1 ms).

The 152 student runs used 95 compute-hours, 45 of them in the 54 runs of the `pqk` family. A further 23 hours went into runs discarded after pipeline fixes. The circuit adds simulation to every training step of a `pqk` run, and none of it remains at deployment, where all methods give models of identical size and speed.

VI. Conclusion

We asked whether a quantum kernel used only during training helps a lightweight waste classifier beyond what a classical kernel in the same place provides. We tested this on TrashNet, with RPS-25 as a low-data confirmation, using three students and five methods under one pipeline. A projected quantum kernel was not distinguishable from logit KD in accuracy. It was ahead on the mean in 3 of 6 pairs and behind on both TrashNet MobileNets, and no paired difference was significant. At its pre-specified scale it beat its RBF twin in 2 of 6 pairs. Its seed-to-seed variance tended to be lower than the twin's and, less consistently, than that of hard-label training. This tendency is not significant at three seeds, depends on the metric and on one mismatched run, and does not hold on every pair. Removing the circuit's entanglement hurt on TrashNet validation and helped on RPS-25, and fitting the teacher projection did not help. All 66 exported MobileNet students are fully integer INT8 models of about 0.62 and 1.21 MB. They run in 2.5 and 3.2 ms per inference on a Raspberry Pi 4B, and the choice of INT8 kernels on the device changes between 0.3% and 9.4% of a model's predictions. The quantum kernel leaves nothing in the deployed model, and our evidence does not show an accuracy benefit over classical alternatives.

These results are exploratory. One question remains open: whether the quantum map beats classical maps of equal width. Answering it would need a frozen protocol, the random-feature control we did not run, at least five seeds and fresh data. On the deployment side, energy and further devices remain to be measured.

References

[1] M. Yang and G. Thung, "Classification of trash for recyclability status," 2016, cS229 project report, Stanford University. TrashNet dataset.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.

[3] A. Howard *et al.*, "Searching for MobileNetV3," in *IEEE/CVF International Conference on Computer Vision*, 2019.

[4] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[6] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational knowledge distillation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3967–3976.

[7] Y. Tian, D. Krishnan, and P. Isola, "Contrastive representation distillation," in *International Conference on Learning Representations*, 2020.

[8] F. Paparounas *et al.*, "Evaluating knowledge distillation and compression techniques for edge class devices," 2024.

[9] C.-Y. Liu, K.-C. Chen, K. Murota, S. Y.-C. Chen, and E. Rinaldi, "Quantum relational knowledge distillation," 2025, preprint.

[10] M. Li, L. Fan, A. Cummings, X. Zhang, M. Pan, and Z. Han, "Hybrid quantum classical machine learning with knowledge distillation," in *IEEE International Conference on Communications*, 2024.

[11] M. J. Hasan and M. R. C. Mahdy, "Bridging classical and quantum machine learning: Knowledge transfer from classical to quantum neural networks using knowledge distillation," arXiv:2311.13810, 2023.

[12] S. R. Naqvi, Z. Ding, F. Qi, M. M. Rahman, and L. Peng, "Structured supervision from quantum models: Distilling robustness into classical networks," in *AAAI Spring Symposium Series*, 2026.
[13] S. Thanasilp *et al.*, "Exponential concentration in quantum kernel methods," *Nature Communications*, vol. 15, 2024.
[14] L. Slattery *et al.*, "Numerical evidence against advantage with quantum fidelity kernels on classical data," *Physical Review A*, vol. 107, p. 062417, 2023.
[15] R. Flórez-Ablan *et al.*, "On the similarity of bandwidth-tuned quantum kernels and classical kernels," *Quantum Science and Technology*, 2025.
[16] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Communications*, vol. 12, p. 2631, 2021.
[17] L. Moroney, "Rock, paper, scissors dataset," 2019, https://laurencemoroney.com/datasets.html.
[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
[19] B. He *et al.*, "Feature kernel distillation," 2022.
[20] X. Wang *et al.*, "Towards understanding the power of quantum kernels in the NISQ era," *Quantum*, vol. 5, p. 531, 2021.
[21] J. M. Kübler *et al.*, "The inductive bias of quantum kernels," in *Advances in Neural Information Processing Systems*, 2021.
[22] J. Schnabel and M. Roth, "Quantum kernel methods under scrutiny: a benchmarking study," *Quantum Machine Intelligence*, 2025.
[23] S. Kumar *et al.*, "Enabling edge intelligence: A quantum-inspired approach to deploying AI models on resource-constrained devices," in *ICEARS*, 2026.
[24] J. Violos, S. Papadopoulos, and Y. Kompatsiaris, "Towards optimal trade-offs in knowledge distillation for CNNs and vision transformers at the edge," arXiv:2407.12808, 2024.
[25] Y. Matsubara *et al.*, "Head network distillation: Splitting distilled deep neural networks for resource-constrained edge computing systems," *IEEE Access*, 2020.
[26] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," 2018.
[27] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020.